PseudoPy

Automated Code Generation System

SYSTEM TEST SUITE SPECIFICATION
(Verification & Evaluation)

Target System: PseudoPy Web Platform
Focus Modules: Authentication, Exercises, Assessments, Compiler, Metrics
Environment: Vanilla JS Frontend + IndexedDB + Skulpt Python Engine
Seeded Test Accounts: admin, instructor, student

This test suite specifies the operational tests, boundary cases, and compiler translation assertions required to verify the correctness of the PseudoPy automated system.

1. Document Overview

This document details the test suite for the PseudoPy application. The test suite verifies user authentication flows, instructor assessment creation, student exercise execution, compiler accuracy, and performance benchmarking.

Testing is divided into five core components:
1. Authentication & Role-Based Access Control (TC_AUTH)
2. Assessment & Exercise Management (TC_ASS)
3. Compiler Translation Engine (TC_COMP)
4. Performance Metrics & Benchmarking (TC_METR)
5. PWA Caching & Offline Usability (TC_PWA)

2. Seeded Test Credentials

The local IndexedDB instance seeds the following users on system initialization. These credentials must be used for all role-based verification tests.

Username: mbautista_admin	**Password: admin123**	**Role: Admin**	(Mark Bautista)
Username: mreantaso_instructor	**Password: pass123**	**Role: Instructor**	(Marc Reantaso)
Username: emirandilla_student	**Password: pass123**	**Role: Student**	(Eduard Mirandilla)
Username: mdaet_student	**Password: pass123**	**Role: Student**	(Mikaella Daet)

3. Authentication & Access Control (TC_AUTH)

TC_AUTH_01 Valid Administrator Login

Description: Verify that an admin can log in and access administrative features.
Preconditions: System is loaded, database is seeded, user is on the login screen.

TEST STEPS

1. Enter username 'mbautista_admin' and password 'admin123'.
2. Click the 'Sign In' button.

EXPECTED RESULT

System parses username suffix, determines 'admin' role, displays Admin panel, and shows a welcome success toast.

TC_AUTH_02 Valid Instructor Login

Description: Verify that an instructor can log in and access learning analytics & exercises.
Preconditions: System is loaded, database is seeded, user is on the login screen.

TEST STEPS

1. Enter username 'mreantaso_instructor' and password 'pass123'.
2. Click the 'Sign In' button.

EXPECTED RESULT

System parses suffix, determines 'instructor' role, displays Instructor panel (Analytics, Manage Exercises, Generate Code).

TC_AUTH_03 Valid Student Login

Description: Verify that a student can log in and access the code workspace & assignments.
Preconditions: System is loaded, database is seeded, user is on the login screen.

TEST STEPS

1. Enter username 'emirandilla_student' and password 'pass123'.
2. Click the 'Sign In' button.

EXPECTED RESULT

System parses suffix, determines 'student' role, displays Student panel (Write Pseudocode, Suggestions, Exercises).

TC_AUTH_04 Invalid Credentials Handling

Description: Verify that logging in with incorrect credentials fails gracefully.
Preconditions: User is on the login screen.

TEST STEPS

1. Enter username 'emirandilla_student' and incorrect password 'wrong123'.
2. Click 'Sign In'.

EXPECTED RESULT

Login fails. A red toast notification reading 'Invalid username or password' is shown. User remains on login page.

TC_AUTH_05 Role-Based Navigation Enforcements

Description: Ensure students cannot access instructor/admin views via console calls or manual redirection.
Preconditions: Student 'emirandilla_student' is logged in.

TEST STEPS

1. Attempt to execute `navigateTo('analytics')` or `navigateTo('manage-exercises')` in the browser console.

EXPECTED RESULT

The navigation handler catches unauthorized calls. Non-student page elements remain hidden and blocked.

4. Assessment & Exercise Management (TC_ASS)

TC_ASS_01 Create New Pseudocode Exercise

Description: Verify that an Instructor can successfully create and publish code exercises.
Preconditions: Instructor 'mreantaso_instructor' is logged in and on the 'Manage Exercises' view.

TEST STEPS

1. Click 'Add New Exercise'.
2. Fill out Title, Concept, Description, Starting Pseudocode, and target Python.
3. Click 'Save Exercise'.

EXPECTED RESULT

Exercise is validated, saved into the IndexedDB 'pseudopy_exercises' object store, and appended to the exercise list.

TC_ASS_02 View Exercises List and Progress States

Description: Verify that students see correct assignment status badges (Not Started, In Progress, Completed).
Preconditions: Student 'emirandilla_student' is logged in and has completed exercise 'algo_1'.

TEST STEPS

1. Navigate to the 'Exercises & Tasks' view.

EXPECTED RESULT

The exercises list displays 'algo_1' with a green 'Completed' badge, and other exercises as 'Not Started' or 'In Progress'.

TC_ASS_03 Solve Exercise and Solution Evaluation

Description: Verify that submitting a solved exercise accurately logs progress and records success metrics.
Preconditions: Student has selected an active exercise and written the matching pseudocode.

TEST STEPS

1. Run the compilation tests.
2. When matching results are returned, click 'Submit Solution'.

EXPECTED RESULT

The solution is compared to target outcomes, a submission entry is pushed to the 'activity' log, and success metrics update.

TC_ASS_04 Automated Structural Suggestion System

Description: Ensure the system flags unclosed blocks (IF/FOR/WHILE) and misspelled keywords.
Preconditions: Student is on the 'Write Pseudocode' editor space.

TEST STEPS

1. Input pseudocode with an unclosed statement:

```
BEGIN
  IF x > 5 THEN
    PRINT x
END
```

2. Press 'Translate Code'.

EXPECTED RESULT

The parser flags the unclosed IF block, blocks compilation, and renders a yellow warning suggestion: 'Add END IF to close this block.'

TC_ASS_05 Access Learning Analytics Dashboard

Description: Verify that instructors can view compilation, success, and error trends.
Preconditions: Instructor is logged in.

TEST STEPS

1. Navigate to the 'Learning Analytics' panel.

EXPECTED RESULT

A visual statistics grid displays Total Translations, Compilation Success Rate, and detailed student submissions table from IndexedDB.

5. Compiler Translation Engine (TC_COMP)

TC_COMP_01 Variable Declarations and Simple Datatypes

Description: Ensure variables are parsed and mapping formats are followed.

INPUT PSEUDOCODE

```
BEGIN
  DECLARE limit AS INTEGER
  limit = 50
  PRINT limit
END
```

EXPECTED PYTHON GENERATION

```
limit = 50
print(limit)
```

ASSERTION

Variable name is added to symbol table, type checks out, and parses into python.

TC_COMP_02 Conditional Branching Structures

Description: Verify translation of IF-THEN-ELSE blocks.

INPUT PSEUDOCODE

```
BEGIN
  IF x > 0 THEN
    PRINT "Positive"
  ELSE
    PRINT "Zero/Negative"
  ENDIF
END
```

EXPECTED PYTHON GENERATION

```
if x > 0:
    print("Positive")
else:
    print("Zero/Negative")
```

ASSERTION

Translates to Python nested block with correct colon (:) delimiters and 4-space indentations.

TC_COMP_03 Range-Based For Loops & Increments

Description: Verify compilation of FOR-FROM-TO-DO iterations.

INPUT PSEUDOCODE

```
BEGIN
  FOR i FROM 1 TO 10 DO
    PRINT i
  ENDFOR
END
```

EXPECTED PYTHON GENERATION

```
for i in range(1, 10 + 1):
    print(i)
```

ASSERTION

Compiled Python uses native range mapping where the end range is inclusive (+1).

TC_COMP_04 Modulo Arithmetic Operators

Description: Verify MOD keyword mapping to % operator.

INPUT PSEUDOCODE

```
BEGIN
```

```
  IF x MOD 2 == 0 THEN
    PRINT "Even"
  ENDIF
END
```

EXPECTED PYTHON GENERATION

```
if x % 2 == 0:
    print("Even")
```

ASSERTION

MOD token matches the compiler keyword pattern and compiles directly into %.

TC_COMP_05 Variable-Sensitive Input Prompt Casting

Description: Verify that input variable names dynamically control casting functions.

INPUT PSEUDOCODE

```
BEGIN
  INPUT WITH PROMPT "Enter user: " username
  INPUT WITH PROMPT "Enter age: " userAge
END
```

EXPECTED PYTHON GENERATION

```
username = input("Enter user: ")
userAge = float(input("Enter age: "))
```

ASSERTION

'username' (containing 'name') compiles as string input. 'userAge' compiles with float() casting wrapper.

6. Performance Metrics & Benchmarking (TC_METR)

TC_METR_01 In-Browser Skulpt Execution

Description: Verify compiled Python execution works directly in frontend using Skulpt.

TEST STEPS

1. Translate pseudocode that prints output.
2. Click the 'Run Python' execution button.

EXPECTED RESULT

Skulpt interpreter loads, executes Python script in virtual sandbox, and appends output logs to console canvas.

TC_METR_02 Automated Benchmark Suite (10,000 Records)

Description: Evaluate translation precision, recall, and compiled correctness against ground truth database.

TEST STEPS

1. Log in as Instructor.
2. Open 'Compiler Metrics & Evaluation' tab.
3. Click 'Run Benchmark' button.

EXPECTED RESULT

System loads 10,000 cases from dataset.json. Compiles each program asynchronously, calculates accuracy, precision, and averages, and renders performance timing charts.

7. PWA Caching & Offline Usability (TC_PWA)

TC_PWA_01 Offline Page Loading and Execution

Description: Verify service worker caching allows full offline usage.

TEST STEPS

1. Load site once to install Service Worker.
2. Turn off network connection / set to Offline in DevTools.
3. Refresh page.

EXPECTED RESULT

Site successfully loads static resources (styles, script engines, database) from cache. Input and compilation engines remain functional.

TC_PWA_02 PWA Installation Prompt Dialog

Description: Verify the custom app shell triggers standard PWA install dialog.

TEST STEPS

1. Visit site using Chrome or Safari browser.
2. Select 'Add to Home Screen' or click PWA badge.

EXPECTED RESULT

System listens to 'beforeinstallprompt' event, displays install button, and opens app in standalone window when selected.